AI-Powered Fire Prediction Spread and Containment System

System Testing Policy

Panic! At The Kernel
Client: EPI-USE Africa

October 2026



Full Name	Student Number
Megan Lai	u23608821
Inge Keyser	u23544563
Janri du Toit	u23632730
Ryan Lynn	u23541912
Marco de Wit	u22811975

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3

2 Testing Plan Structure **3**
2.1 Types of Testing . . . 3
2.1.1 Functional Testing . . . 3
2.2 Test Coverage Criteria . . . 3

3 Automated Testing **4**
3.1 CI/CD - GitHub Actions . . . 4
3.2 Testing Framework Selection . . . 4
3.2.1 pytest (Python Testing Framework) . . . 4
3.3 Playwright (End-to-end Testing) . . . 4

4 Functional Testing **4**
4.1 Unit Testing . . . 4
4.1.1 API layer Testing . . . 4
4.1.2 AI Model Unit Testing . . . 5
4.2 Integration Testing . . . 5
4.3 End-to-end Testing . . . 6

5 Test Reports and Documentation **6**
5.1 Automated Test Reports . . . 6

6 Change Log **7**

1 Introduction

This document sets out the standards and responsibilities for testing for Fire Away - an AI-Powered Fire Spread Prediction and Containment System - so that testing is planned, executed, and documented consistently, and the software meets the expected quality and reliability requirements.

1.1 Purpose

The purpose of this document is to:

- Catch defects early to ensure they do not reach production
- Prevent regressions as our codebase evolves
- Keep testing practices consistent across contributors and components

1.2 Scope

This policy applies to all code in the project, including:

- Frontend components
- Backend/API services
- AI/ML model
- Database interactions

It applies across the full development lifecycle - from local development through our CI/CD to deployment - and to all contributors.

2 Testing Plan Structure

2.1 Types of Testing

2.1.1 Functional Testing

Functional testing verifies that our system behaves correctly against all the requirements we set our in our system requirements specification document. It covers unit, integration, and end-to-end (system) testing, which will be detailed in Section 4.

2.2 Test Coverage Criteria

- **Code Coverage:** Minimum 80% statement and branch coverage.
- **Use case coverage:** All use cases that are implemented must be tested.
- **API Coverage:** All endpoints need to be validated.
- **Requirements Coverage:** Each functional requirement must be tested.

3 Automated Testing

3.1 CI/CD - GitHub Actions

GitHub Actions is our chosen CI/CD platform for our project. It eliminates the friction between where code is stored and where it is tested and deployed.

Benefits:

- Supports native integration and no setup overhead
- Supports event-driven triggers beyond just Git
- Free to use for public repositories

Our CI/CD pipeline is triggered on every pull request. It automatically runs linting, the full pytest suite, Playwright end-to-end tests and coverage reports.

3.2 Testing Framework Selection

3.2.1 pytest (Python Testing Framework)

- Used for unit and integration testing across our codebase
- Used to test our AI Model as well
- Provides test running, fixtures, parametrization and coverage reporting (via `pytest-cov`)
- Provides a rich plugin ecosystem

3.3 Playwright (End-to-end Testing)

- Used for end-to-end testing of user facing flows in the browser
- Provides native integration with pytest
- Generates full interactive traces of test runs
- Includes built-in device emulation, allowing testing for mobile viewpoints

4 Functional Testing

4.1 Unit Testing

4.1.1 API layer Testing

Location: `app/backend/src/tests/*_unit.py`

Test Categories:

- **Data Validation:** Verify schema enforcement, validenums, required fields, correct data dypes
- **Business Logic:** Test state transitions, default value assignments, domain-specific rule enforcement
- **Endpoint Behaviour:** Validate accurate HTTP status codes, correct data retrieval, empty state handling

- **Authentication & Security:** Validate cryptographic hashing along side token creation, encoding, expiration logic
- **Error Handling:** Verify the system raises expected exceptions and returns appropriate HTTP error codes for invalid actions or missing records

```python
def test_revoke_with_no_matching_user(self):
        """User lookup returns None. Raise 404 instead of silent succeed""
            "
        req = make_request(status = "approved")
        db = query_side_effect(make_db(), {roleRequestDB: req, _User: None
            })

        with pytest.raises(HTTPException) as exc:
            revoke_role_request("req-1", db = db)

        assert exc.value.status_code == 404
```

Example: Python Unit Test

4.1.2 AI Model Unit Testing

Location: `app/backend/testing/unit/*_unit.py`

Test Categories:

- **Schema & Constant Validation:** Verify core feature lists combine correctly without duplicate column names, ensures correct integer mappings for categorical states
- **Data Transformation:** Multidimensional environmental grids correctly flattened and formatted

```python
def test_downslope_burning_lower_neighbour(small_grids):
    """Cell uphill of burning cell should see downslope_burning = 1 """
    weather, static, burn = small_grids()
    static["elevation"][2, 2] = 400.0
    burn[2, 2] = BURNING
    nbf = neighbour_features(burn, weather["wind_u"], weather["wind_v"],
        static["elevation"])
    assert nbf["downslope_burning"][2, 3] == pytest.approx(1.0)
```

Example: AI Model Python Unit Test

4.2 Integration Testing

Location: `app/backend/testing/integration/*_integration.py`

Test Scenarios:

- Authentication and Access Control
- Admin role lifecycle management
- Database persistence and retrieval
- Schema validation and error handling

- Data privacy and public exposure

```python
def test_get_reports(client, db):
    report = make_report(db)
    response = client.get("/api/users/reported-fires")

    assert response.status_code == 200
    data = response.json()
    assert isinstance(data, list), "must return list"
    assert len(data) >= 1, "must return atleast 1 report"

    refnums = []
    for r in response.json():
        refnums.append(r["reference_number"])
    assert report.reference_number in refnums, " report not found in get response"
```

Example: Integration Test

4.3 End-to-end Testing

Location: `app/frontend/testing`

Primary Test Flows:

- User Authentication Journey
- Reporting a Fire Journey
- Admin Role Approval Journey
- Firefighter Dashboard functionality Journey

5 Test Reports and Documentation

5.1 Automated Test Reports

Code coverage reports are automatically generated after running all tests as part of the CI/CD workflow.

- **Python code coverage report:** Uploads as a comment in the respective pull request.
- **Frontend E2E code coverage:** Uploads to GitHub Actions Artifacts.

6 Change Log

Date	Version	Change
July 2026	1.0	Initial version (Demo 2)
October 2026	2.0	Update testing folder directories